

개 발 기 록

하루마을 개발 기록

일기 기록 + 캐릭터 수집 + 공간 꾸미기를 결합한
2.5D 캐주얼 생활 게임 · PHASE 1 - 2

프로젝트	diary (하루마을)
저장소	github.com/rmsdnlggh00/diary
기술	React 19 · TypeScript · Vite 6 · Zustand (Unity 미사용, 웹 기술만)
진행 단계	PHASE 1 완료 · PHASE 2 완료
작성일	2026년 9월 22일

1. 프로젝트 개요

"내가 하루를 기록하면 그날의 내가 게임 속 캐릭터가 되어 작은 마을에 남는다."

매일 일기를 쓰면 그 날짜의 캐릭터가 생성되어 마을에 하나씩 쌓인다. 일기의 분위기에 따라 캐릭터의 표정이 달라지고, 사용자는 마을을 자기 취향대로 꾸밀 수 있다.

핵심 규칙

- 하루에 캐릭터는 최대 1개. 9월이면 최대 30명, 10월이면 최대 31명.
- 같은 날짜의 일기를 수정하면 새 캐릭터를 만들지 않고 기존 캐릭터의 감정만 갱신한다.
- 감정은 6종 — 행복(HAPPY) · 즐거움(JOYFUL) · 보통(NORMAL) · 우울(SAD) · 화남(ANGRY) · 짜증(ANNOYED).
- 감정은 게임 연출용이며, 실제 정신건강을 진단하는 기능이 아니다.
- 월드는 월 단위로 바뀌고, 각 월의 캐릭터 데이터는 월별로 유지된다.

기술 선택

항목	선택	이유
빌드	Vite 6	빠른 HMR. Node 22.9 환경이라 Node 22.12 이상을 요구하는 Vite 8 대신 6으로 고정했다.
프레임워크	React 19 + TypeScript	레이어 · 컴포넌트 구조와 잘 맞고 타입으로 데이터 구조를 고정할 수 있다.
상태관리	Zustand 5	데이터(store)와 UI(컴포넌트)를 완전히 분리하기 위해.
렌더링	DOM + CSS transform	Canvas 대신 DOM을 써서 클릭 처리, UI 겹침, 에셋 교체가 쉽다. 2.5D는 scale과 z-index로 만든다.
저장	localStorage (예정)	서버·DB·로그인 없이 시작한다.

설계 원칙 3가지

- 좌표는 전부 **normalized(0~1)** — 픽셀을 저장하지 않으므로 창 크기가 바뀌어도 위치가 유지된다.
- systems/** 는 순수 **TypeScript** — React를 import하지 않는다. 그래서 감정 분석기를 통째로 AI API로 교체할 수 있다.
- 에셋은 레지스트리를 통해서만 접근 — 컴포넌트는 이미지 경로를 알지 못한다.

2. PHASE 1 — 게임처럼 보이는 한 화면

목표는 "실행했을 때 게임처럼 보이는 하나의 화면"이었다. 실제 일기 저장, 상점 구매, 인벤토리, AI 분석은 의도적으로 만들지 않고 구조만 남겼다.

구현한 것

- 16:9 비율을 유지하는 게임 화면(stage). 내부 요소는 CSS 컨테이너 쿼리 단위(`cqw`)를 써서 화면 크기에 함께 맞춰진다.
- 하늘 · 언덕 · 지면으로 나뉜 배경 placeholder.
- 상점 · 연못 · 다리 · 길 · 울타리 · 큰 나무 등 정적 환경 placeholder.
- 레이어로 합성되는 캐릭터 placeholder (몸 · 표정 · 머리 · 옷 · 액세서리).
- 상단 UI(월 · 오늘의 기분 · 보유 코인), 하단 UI(오늘 기록 · 상점 · 캐릭터 꾸미기 · 공간 꾸미기).
- y 좌표 기반 깊이 시스템 — 앞쪽일수록 커지고 앞에 그려진다.
- 캐릭터를 클릭하면 그 날짜의 일기 상세가 열린다.



PHASE 1 실행 화면 — 배경, 정적 환경, 캐릭터 5명, 상·하단 UI

깊이(2.5D) 시스템

실제 3D를 쓰지 않는다. y 좌표 하나로 크기와 렌더 순서를 동시에 계산해서 깊이감을 만든다. 모든 월드 오브젝트는 예외 없이 이 계산을 거친다.

```
// src/systems/depthSystem.ts
const DEFAULT_DEPTH_CONFIG = {
  horizonY: 0.46, // 이 위쪽은 원경
  frontY: 1.0, // 화면 맨 앞
  minScale: 0.72,
  maxScale: 1.18,
}
```

```
getDepthScale(y) → 뒤쪽 0.72 ... 앞쪽 1.18  
getRenderOrder(y) →  $y * 10000$  을 그대로 z-index 로 사용
```

감정 분석기 — 교체 가능한 독립 모듈

UI는 감정 분석기를 직접 호출하지 않는다. 지금은 키워드 규칙 기반이지만, `analyzeDiary` 시그니처만 지키면 AI API 구현으로 통째로 갈아끼울 수 있다.

```
// src/systems/emotionAnalyzer.ts  
export type DiaryAnalyzer = (text: string) => EmotionResult  
  
analyzeDiary("오늘 정말 행복했다")  
  → { emotion: "HAPPY", confidence: 0.7 }
```

PHASE 1 결과는 커밋 `daa2c76` 으로 저장소에 올라가 있다.

3. PHASE 2 — 실제 에셋을 넣을 수 있는 월드 구조

PHASE 2의 목표는 새 기능을 늘리는 것이 아니라, PNG/WebP 파일을 폴더에 넣기만 하면 코드 수정 없이 게임에 반영되는 구조를 완성하는 것이었다. PHASE 1의 게임 로직은 다시 만들지 않고 그대로 유지했다.

3.1 Asset Registry

이미지 경로를 컴포넌트 여기저기에 하드코딩하지 않는다. 경로는 레지스트리 한 곳에만 있고, 컴포넌트는 조회 함수만 부른다.

```
src/data/assets/
├─ index.ts           조회 함수만 노출 (컴포넌트는 경로를 모름)
├─ worldAssets.ts     월별 background + environment
├─ decorationAssets.ts BENCH / TREE / FLOWER / BICYCLE / LAMP / PICNIC_MAT
├─ npcAssets.ts       DOG
├─ characterAssets.ts body / emotions / hair / outfits / accessories
```

3.2 에셋 메타데이터와 anchor

이미지마다 "바닥에 닿는 지점"이 다르다. 나무는 밑동, 캐릭터는 발, 벤치는 다리다. 중앙을 기준으로 배치하면 땅에 박히거나 공중에 뜬다. 그래서 에셋마다 anchor를 갖는다.

```
{
  id: "decoration.bench",
  type: "BENCH",
  src: "/assets/decorations/bench/bench.webp",
  anchorX: 0.5,      // 가로 중앙
  anchorY: 0.9,      // 세로 90% 지점이 바닥
  defaultScale: 1,
  worldWidth: 0.08,  // 월드 가로 대비 폭
  layer: "decoration"
}
```

그림자도 이 anchor 지점에 붙기 때문에, 에셋을 교체해도 접지가 어긋나지 않는다.

3.3 placeholder → 실제 에셋 자동 전환

Sprite 컴포넌트가 이 판단을 전담한다. 레지스트리에 적힌 경로에 파일이 있으면 이미지를, 없으면 도형 placeholder를 그린다. 파일 존재 확인은 경로당 한 번만 하고 캐싱한다.

Asset Registry → 파일 존재 확인 → 있으면 / 없으면 placeholder 도형

3.4 Walkable / Blocked Area

복잡한 물리 충돌 대신 사각 영역 데이터만으로 관리한다. 캐릭터는 walkable 영역 안에서만, blocked 영역과 다른 오브젝트를 피해 서 생성된다. 연못 위나 건물 안, 화면 밖에 서는 일이 없다.

```
walkableAreas: [ lawn-back, lawn-mid, lawn-front ]  
blockedAreas: [ pond, shop, tree-front-right, tree-left ]  
  
// 하단 UI에 가리지 않도록 walkable 영역의 y는 0.85까지만 사용
```

3.5 캐릭터 레이어 분리

PHASE 1에서는 하나의 SVG 안에 그룹으로 나뉘어 있었는데, PHASE 2에서 레이어마다 독립된 요소로 분리했다. 이제 표정만, 옷만 따로 이미지로 교체할 수 있다. 옷을 바꿔도 그 날씨의 감정 표정은 그대로 유지된다.

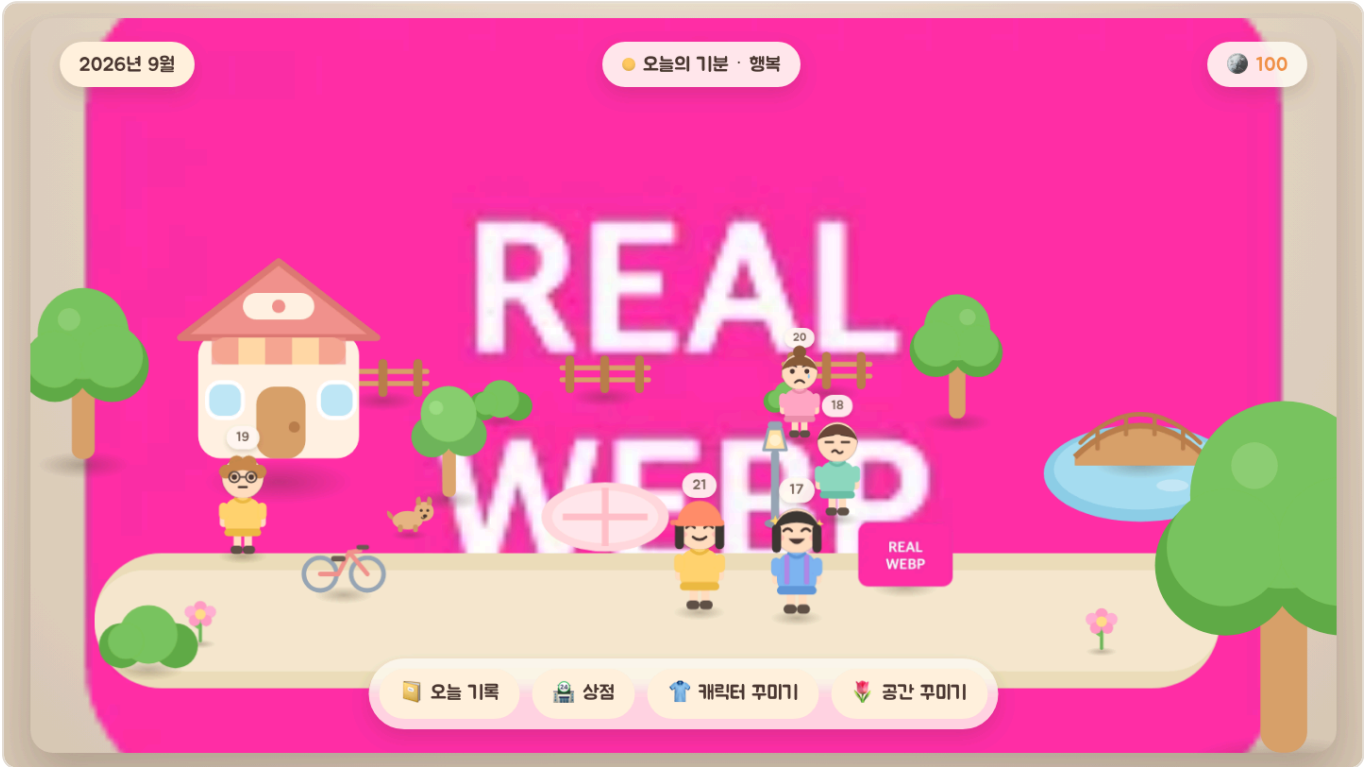
```
Character  
├─ Body      characters/body/{id}.webp  
├─ Outfit    characters/outfits/{id}.webp  
├─ Face      characters/emotions/{happy|joyful|normal|sad|angry|annoyed}.webp  
├─ Hair      characters/hair/{id}.webp  
└─ Accessories characters/accessories/{id}.webp
```

4. 검증 결과

Chrome을 실제로 띄워 클릭·리사이즈까지 수행하며 확인했다. 눈으로 본 것뿐 아니라 계산값을 직접 읽어서 검사했다.

4.1 에셋 교체 테스트

테스트용 WebP 파일을 `public/assets` 아래에 넣었다가 지우는 방식으로 확인했다. 코드는 한 줄도 수정하지 않았다.



배경 파일 하나만 넣었을 때 — 배경만 이미지로 바뀌고 상점 · 나무 · 강아지 · 장식물 · 캐릭터는 모두 독립 객체로 그대로 유지된다. 오른쪽 아래 분홍 사각 형은 벤치 자리에 넣은 테스트 이미지다.

4.2 깊이 시스템 수치 검증

화면에 있는 월드 오브젝트 24개 전부의 계산값을 읽어 확인했다.

항목	결과
검사 대상	월드 오브젝트 24개 (환경 · 장식물 · NPC · 캐릭터)
z-index	y가 커질수록 단조 증가 — 앞쪽 객체가 항상 위에 그려짐
scale	y가 커질수록 단조 증가 — 0.758 (뒤) … 1.18 (앞)
화면 비율	1.778 (16:9). 창을 900×1000으로 줄여도 유지됨
콘솔 에러	0건

4.3 PHASE 2 완료 조건 점검

✓ 16:9 게임 화면이 정상 출력된다

- ✓ Background가 에셋으로 로드된다
- ✓ 정적 환경이 독립된 에셋으로 렌더링된다
- ✓ 벤치 · 나무 · 꽃 · 자전거가 독립된 에셋으로 렌더링된다
- ✓ Asset Registry가 이미지 경로를 일괄 관리한다
- ✓ 이미지가 없으면 placeholder가 나타난다
- ✓ 모든 오브젝트가 anchor point를 지원한다
- ✓ y 위치에 따라 scale이 변한다
- ✓ y 위치에 따라 z-index가 변한다
- ✓ 캐릭터가 배경보다 앞/뒤에 자연스럽게 나타난다
- ✓ Walkable Area와 Blocked Area 구조가 존재한다
- ✓ 2026-09 월드 데이터를 독립적으로 관리한다
- ✓ 2026-10 월드를 추가하기 쉽다 (데이터 파일에 키 하나씩만 추가)
- ✓ 실제 에셋을 넣으면 코드 수정 없이 게임에 나타난다



PHASE 2 최종 화면 — 상점 · 연못 · 다리 · 울타리(환경), 나무 · 매트 · 가로등 · 벤치 · 자전거 · 꽃(장식물), 강아지(NPC), 캐릭터 5명

5. 전체 파일 구조

```
src/
├── components/
│   ├── game/
│   │   ├── GameWorld.tsx          레이어 조립 + 깊이 정렬
│   │   ├── Background.tsx        하늘 / 언덕 / 지면
│   │   ├── StaticEnvironment.tsx  상점 · 연못 · 다리 · 길 · 울타리 · 큰 나무
│   │   ├── PlaceableObject.tsx   사용자 장식물
│   │   ├── NPC.tsx               강아지
│   │   ├── DiaryCharacter.tsx     일기 캐릭터
│   │   ├── WorldObject.tsx       좌표 · anchor · 깊이 적용 (모든 객체의 공통 껍데기)
│   │   ├── Sprite.tsx            에셋 있으면 이미지 / 없으면 placeholder
│   │   ├── character/
│   │   └── placeholders/         종류별 임시 도형
│   ├── diary/DiaryDetail.tsx
│   └── ui/                      TopBar · BottomBar · PanelSheet
├── data/
│   ├── assets/                  Asset Registry
│   ├── worlds.ts               2026-09 월드 정의
│   ├── emotions.ts             감정 6종
│   ├── characterParts.ts        몸 · 머리 · 옷 · 액세서리
│   └── mockCharacters.ts        PHASE 1~2 확인용 더미 (일기 시스템 붙으면 제거)
├── systems/
│   ├── depthSystem.ts           React를 import하지 않는 순수 로직
│   │   └── y → scale, z-index
│   ├── emotionAnalyzer.ts       일기 → 감정 (AI로 교체 예정)
│   ├── characterSystem.ts       스폰 위치 결정
│   ├── worldSystem.ts           walkable / blocked 판정
│   └── assetSystem.ts           에셋 존재 확인 + 캐싱
├── store/gameStore.ts           Zustand
├── hooks/                       React 어댑터
├── types/                       전체 데이터 타입
└── utils/

public/assets/
├── worlds/2026-09/{background, environment}/
├── characters/{body, emotions, hair, outfits, accessories}/
├── decorations/{bench, tree, flower, bicycle, lamp, picnic}/
├── npcs/dog/
└── ui/
```

레이어 순서

데이터상 레이어는 배경 · 환경 · 장식물 · NPC · 캐릭터로 나뉘지만, 화면에서는 캐릭터가 나무 앞뒤로 자연스럽게 들어가야 한다. 그래서 세워지는 오브젝트는 하나의 컨테이너에 모아 y 기준으로 함께 정렬한다. 길과 연못처럼 바닥에 깔리는 것만 별도의 지면 레이어에 둔다.

배경 → 바닥 장식(길 · 연못) → [환경 · 장식물 · NPC · 캐릭터 를 y로 함께 정렬] → 이펙트 → UI

6. 실행 방법과 다음 단계

실행

```
npm install
npm run dev      # 개발 서버
npm run build    # 타입 검사 + 프로덕션 빌드
```

빌드 결과: JS 254.97 kB (gzip 79.75 kB), CSS 6.02 kB (gzip 1.96 kB). 현재 5173 포트는 다른 프로세스가 쓰고 있어 개발 서버는 5174로 떠 있다.

에셋을 넣는 방법

레지스트리에 적힌 경로에 파일을 넣기만 하면 된다. 예를 들어 벤치 이미지는 `public/assets/decorations/bench/bench.webp` 에 두면 placeholder가 자동으로 교체된다. 파일이 아직 없는 경로는 최초 1회 404 요청이 발생하는데, 이는 "코드 수정 없이 교체" 구조의 대가이며 실제 에셋이 채워지면 사라진다.

아직 만들지 않은 것

의도적으로 제외한 항목이다. AI 감정 분석, 실제 일기 저장, 상점 구매, 인벤토리, 캐릭터 꾸미기, 장식물 배치 UI, NPC 행동 로직, 서버 DB, 로그인, 결제, 멀티플레이어.

다음 단계 후보

단계	내용
일기 시스템	일기 작성 화면, localStorage 저장, 감정 분석 연결, 날짜별 캐릭터 생성 및 수정
NPC 행동	IDLE · LOOK_AROUND · WALK · SIT 상태 기계와 지정 영역 내 랜덤 이동
꾸미기	장식물 배치 · 이동 · 삭제 · 회전 · 저장, 캐릭터 의상 착용
상점	코인, 아이템 구매, 인벤토리
월 전환	2026-10 월드 추가와 월별 캐릭터 데이터 유지